A Comparison of Supervised Learning Models on Distinct Datasets for Malware Detection

Mariana Carvalho
2Ai-School of Technology
Polytechnic University of Cávado and Ave (IPCA)
Campus of IPCA, 4750-810 Barcelos,Portugal
0009-0004-1818-8213

Daniel Fernandes
Polytechnic University of Cávado and Ave (IPCA)
Campus of IPCA, 4750-810 Barcelos,Portugal
0009-0002-7839-257X

Óscar R. Ribeiro
2Ai-School of Technology
Polytechnic University of Cávado and Ave (IPCA)
Campus of IPCA, 4750-810 Barcelos,Portugal
oribeiro@ipca.pt

Nuno Lopes
2Ai-School of Technology
LASI – Associate Laboratory of Intelligent Systems, Guimarães
Campus of IPCA, 4750-810 Barcelos, Portugal
nlopes@ipca.pt

***Abstract*—Malware detection remains a critical challenge as cybersecurity threats continue to evolve, particularly with the rise of memory-resident and mobile malware. In this study, we evaluate the performance of three supervised learning models—Logistic Regression, Random Forest, and Decision Tree—across two distinct datasets: CIC-MalMem-2022 (focused on memory-resident malware) and CIC-MalDroid-2020 (targeting Android malware). Our findings show that while all models perform exceptionally well on memory-based malware, Random Forest achieves the highest accuracy for mobile malware classification, highlighting its ability to handle complex patterns. Additionally, hyperparameter tuning further improves model robustness, demonstrating the impact of optimization in challenging detection scenarios. This study provides insights into the strengths and limitations of different models, contributing to more effective machine learning-based malware detection strategies.**

***Index Terms*—Malware, Memory Analysis, Android, Machine Learning.**

I. Introduction

The growing complexity of malware poses a significant and escalating threat to individuals, organizations, and governments. Advancements in technology have equipped attackers with increasingly sophisticated tools, leading to the development of malware that is ever more challenging to detect. Traditional signature-based detection methods, which rely on predefined patterns or characteristics of known malware, are proving insufficient against modern malware that leverages advanced anti-analysis techniques such as obfuscation, encryption, evasion, and packing techniques [1].

Recent trends in malware development have led to the emergence of new categories, including fileless malware that operates entirely in memory and Android malware that targets mobile ecosystems. Fileless malware, for instance, avoids leaving traces on the disk by residing solely in volatile memory, evading traditional detection mechanisms. Similarly, Android malware exploits the openness of the Android ecosystem to infiltrate mobile devices through malicious applications, affecting millions of users worldwide. The increasing prevalence and sophistication of these threats necessitate innovative detection strategies that go beyond static analysis methods.

This study focuses on evaluating the performance of supervised machine learning models for malware classification across two distinct domains. The first domain, volatile memory analysis, is represented by the CIC-MalMem-2022 dataset, which captures detailed insights into the behavior of malware in RAM. The second domain, Android malware detection, is represented by the CIC-MalDroid-2020 dataset, which analyzes malware behavior in mobile environments. By combining these datasets, this research aims to assess the ability of machine learning models to generalize across diverse malware types and environments.

This article is structured as follows: The Related Work section reviews existing approaches to malware detection and highlights the challenges of memory-based and Android malware analysis. The Methodology section details the datasets, preprocessing steps, and machine learning models used in this study. The Results and Discussion section presents the performance metrics of the implemented models and offers insights into the observed trends. Finally, the Conclusion summarizes the findings and outlines future directions for advancing malware detection systems.

II. Related Work

Malware detection has traditionally relied on static and dynamic analysis techniques. Static analysis inspects the structure and code of malware without executing it, making it efficient but limited in detecting obfuscated or polymorphic malware [2]. Dynamic analysis observes the behavior of malware during execution, offering insights into runtime characteristics but often requiring resource-intensive sandboxing environments [3].

Memory analysis is the process of examining volatile memory (RAM) to uncover detailed, real-time data about active processes, system states, and ongoing activities within a computing environment. Unlike static file analysis, which focuses on stored files and their metadata, memory analysis provides a dynamic view of a system's operations. This capability is particularly crucial for detecting in memory or fileless malware, a sophisticated type of malicious software that operates entirely within RAM and leaves no artifacts on the file system. By analyzing volatile memory, investigators can identify injected code, malicious processes, and command-and-control communications that might otherwise evade detection through traditional disk-based or signature-based methods [4]. Using the CIC-MalMem-2022 dataset, studies have demonstrated high precision in detecting fileless and memory-resident malware. For instance, machine learning algorithms applied to this dataset achieved a precision of 96.2% in distinguishing malware from benign processes, showcasing its utility in analyzing volatile memory for cybersecurity applications [5].

Another difficulty arose with the increase of mobile devices, and android has become a primary target for attackers. Android malware detection is the process of identifying and analyzing malicious applications designed to target Android devices, safeguarding user data, privacy, and system integrity. Unlike traditional malware detection methods for desktop systems, Android malware detection often deals with the unique challenges posed by mobile platforms, such as permissions misuse, dynamic code loading, and widespread app distribution channels [4]. Using the CIC-MalDroid-2020 dataset, advanced machine learning approaches have achieved high precision rates, such as 97.46% with Random Forest classifiers when combined with feature selection techniques, demonstrating the effectiveness of leveraging optimized feature sets for Android malware detection [3].

Although existing studies show strong overall classification performance, they often neglect the challenges posed by imbalanced datasets, specifically the difficulty of accurately detecting minority classes. In malware detection, overlooking these less frequent but potentially critical categories can compromise the reliability of the system. This study addresses this issue by emphasizing the need for balanced performance across all classes, not just the majority, and incorporates techniques such as SMOTE to mitigate class imbalance and improve detection rates in underrepresented groups.

III. Methodology

This study adopts a machine learning-based methodology to classify malware across two datasets: CIC-MalMem-2022 and CIC-MalDroid-2020.

The CIC-MalMem-2022 dataset [6], a comprehensive and publicly available dataset designed for advanced malware memory analysis, containing labeled samples of benign and malicious memory activity equally split between the two classes. These samples are categorized into various malware families, including ransomware, spyware, and trojan horses, and were collected from isolated virtual machines running Windows 10 with 2GB memory snapshots. The dataset emphasizes realworld relevance by ensuring a balanced distribution of samples and employing SMOTE (Synthetic Minority Oversampling Technique) to address potential class imbalances. The SMOTE technique improves the classification metrics for the minority classes [7], which can be an advantage over the results of related work.

The CIC-MalDroid-2020 [8], [9] dataset was developed for Android malware detection and classification, providing a comprehensive framework for categorizing Android apps into five distinct categories: Adware, Banking Malware, SMS Malware, Riskware, and Benign applications, Figure 1.

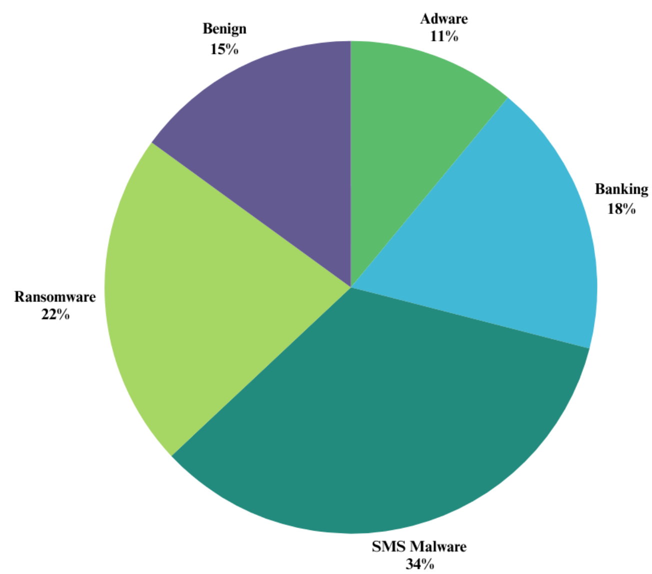

Fig. 1. Malware Type Distribution of CiC-MalDroid-2020 dataset.

It captures both static and dynamic features, including system calls, binder calls, and composite behaviors, which are critical for understanding malware patterns. Preprocessing of the dataset involved detailed feature extraction and normalization, resulting in feature vectors of size 470. These vectors encapsulate key behaviors such as system calls, network access, and binder calls, enabling machine learning models to discern patterns indicative of malicious activity. The diversity of features in the CIC-MalDroid-2020 dataset makes it an appropriate resource for evaluating machine learning approaches in the mobile malware domain. The CIC-MalMem-2022 dataset underwent preprocessing to enable advanced malware memory analysis. It involved splitting malware samples into broader categories - benign, spyware, ransomware, and trojan - and further classifying them into subfamilies, including Zeus, Emotet, Refroso, Scar, Reconyc, 180solutions, CoolWebSearch, Gator, Transponder, TIBS, Conti, MAZE, PYSA, Ako, and Shade, Figure 2.

Following preprocessing, the datasets was divided into 80% for training and 20% for testing, under three models. The

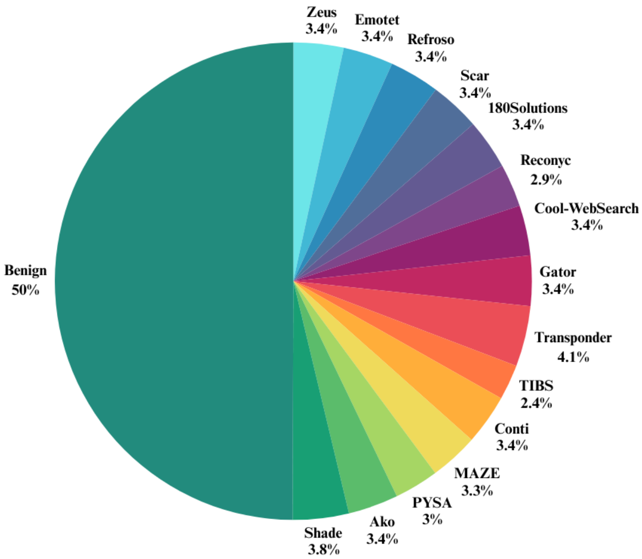

Fig. 2. Malware Type Distribution of CiC-MalMem-2022 dataset.

Logistic Regression (LR) model was employed to assess the linear separability of the data, particularly for binary classification tasks. Random Forest (RF), an ensemble learning model, was utilized to capture complex patterns in the dataset by combining the outputs of multiple decision trees, enhancing robustness and accuracy. Finally, a Decision Tree (DT) model was implemented to provide an interpretable and straightforward classification framework, suitable for understanding feature importance and decision boundaries. To further enhance model performance, grid search methods were applied to optimize hyperparameters such as the regularization strength in Logistic Regression, the number of estimators and maximum depth in Random Forest, and the splitting criteria in Decision Trees. These models collectively enabled comprehensive evaluation and comparative analysis across different machine learning techniques.

IV. Results and Discussion

Table I summarizes the performance metrics of three machine learning models, Logistic Regression (LR), Random Forest (RF), and Decision Tree (DT). Evaluated on the two datasets (CIC-MalMem-2022 and CIC-MalDroid-2020), with the default hyperparameters settings, for metrics such as accuracy, precision, recall, and F1-score were calculated.

All models performed exceptionally well on the CIC-MalMem-2022 dataset, with all models achieving nearly perfect metrics of 99% for all the evaluated metrics. On the other hand, for the CIC-MalDroid-2020 dataset, we can conclude that the Random Forest model outperformed other models, achieving an accuracy and F1-score of 0.9340, highlighting its ability to capture complex relationships in mobile malware patterns. While Logistic Regression lagged behind (F1-score of 0.7459), Decision Tree showed robust performance with an F1-score of 0.8925.

TABLE I
Bening vs. Malware Classification with Default Model Hyperparameters for Both Datasets

Model	Dataset	Accuracy	Precision	Recall	F1-Score
LR	CIC-MalMem-2022	0.9988	0.9988	0.9988	0.9988
	CIC-MalDroid-2020	0.7551	0.7602	0.7551	0.7459
RF	CIC-MalMem-2022	0.9999	0.9999	0.9999	0.9999
	CIC-MalDroid-2020	0.9340	0.9350	0.9340	0.9340
DT	CIC-MalMem-2022	0.9999	0.9999	0.9999	0.9999
	CIC-MalDroid-2020	0.8926	0.8925	0.8926	0.8925

Additional evaluation was performed on the CIC-MalMem-2022 dataset, this time on the model's ability to classify the sub families, the results are shown in Table II.

TABLE II
Sub family classification results for the CIC-MalMem-2022 dataset with default model's hyperparameters

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.7404	0.7436	0.7404	0.7402
Random Forest	0.8700	0.8701	0.8700	0.8700
Decision Tree Classifier	0.8452	0.8453	0.8452	0.8452

These results highlight the superior performance of the Random Forest model in classifying malware subfamilies, achieving the highest accuracy (0.8700) and F1-Score (0.8700) compared to the other models. Logistic Regression demonstrated the lowest performance, emphasizing the importance of model selection for this classification task.

The performance of machine learning models is highly influenced by the characteristics of the dataset. The CIC-MalMem-2022 dataset is well-balanced and features lower-dimensional, structured memory data. This structure simplifies learning, allowing all models, especially linear ones like Logistic Regression, to achieve near-perfect results. On the other hand, the CIC-MalDroid-2020 dataset is imbalanced and high-dimensional, composed mainly of system-level behavioral features such as system and binder calls. These properties increase modeling complexity and reduce linear separability, which explains why Random Forest consistently outperformed the other models on this dataset.

Moreover, computational efficiency plays a crucial role when considering the real-time applicability of malware detection systems. To evaluate this, training and inference times were measured for each model on the CIC-MalMem-2022 dataset. The results revealed that Logistic Regression, despite being the slowest to train (7.88s), offered rapid inference (0.0078ms/sample), making it viable for real-time scenarios after deployment. In contrast, Random Forest had a faster training time (6.20s) but a slower inference rate (0.0158ms/sample), which could impact responsiveness in time-sensitive environments. Decision Tree showed the fastest overall performance, with minimal training time (0.52s) and the fastest inference (0.0003ms/sample), making it particularly efficient. These findings highlight the trade-off between model complexity and execution speed, which is essential for selecting the most suitable algorithm for operational use in cybersecurity applications.

A. Hyperparameter tuning

Randomized Grid Search was then applied to try to improve the model's performance, the tuning process involved exploring a predefined range of hyperparameters for each model to identify the optimal configuration. For Logistic Regression, Table III, hyperparameters such as the number of iterations, regularization strength, solver algorithm, and penalty type, were explored. Similarly, for the Random Forest, Table IV, and Decision Tree models, Table V, the number of estimators, maximum depth, minimum samples for split and leaf were explored.

TABLE III
HYPERPARAMETER RANGES EXPLORED DURING RANDOMIZED GRID SEARCH FOR LOGISTIC REGRESSION.

Hyperparameter	Values
`C`	`loguniform(1e-4, 1e4)`
`penalty`	{`'l1'`, `'l2'`, `'elasticnet'`}
`solver`	{`'liblinear'`, `'saga'`, `'lbfgs'`}
`l1_ratio`	{0.1, 0.5, 0.9}
`max_iter`	{250, 700, 1500}

TABLE IV
HYPERPARAMETER RANGES EXPLORED DURING RANDOMIZED GRID SEARCH FOR RANDOM FOREST.

Hyperparameter	Values
`n_estimators`	{10, 50, 100, 200, 500}
`max_depth`	{None, 10, 20, 30, 40}
`min_samples_split`	{2, 5, 10}
`min_samples_leaf`	{1, 2, 4}
`criterion`	{`'gini'`, `'entropy'`}

TABLE V
HYPERPARAMETER RANGES EXPLORED DURING RANDOMIZED GRID SEARCH FOR DECISION TREE CLASSIFIER.

Hyperparameter	Values
`max_depth`	{None, 10, 20, 30, 40}
`min_samples_split`	{2, 5, 10, 20}
`min_samples_leaf`	{1, 2, 5, 10}
`criterion`	{`'gini'`, `'entropy'`}
`splitter`	{`'best'`, `'random'`}

For the CIC-MalMem-2022 dataset the evaluation was performed regarding the sub family classification, the results are presented in Table VI.

When compared to the results without hyperparameter tuning (Table II), we observe that the Logistic Regression model choose to keep its default solver only changing the values of its regularization strength (C) to 253.69148616733443, and the number of max-iter to 1500 displayed only a slight improvement in accuracy (+0.78%) but experienced a decline

TABLE VI
SUB FAMILY CLASSIFICATION RESULTS FOR THE CIC-MALMEM-2022 DATASET WITH MODEL'S HYPERPARAMETERS TUNING

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.7482	0.6283	0.6248	0.6232
Random Forest	0.8706	0.8067	0.8067	0.8700
Decision Tree Classifier	0.8470	0.7714	0.7712	0.8452

in precision (-11.53%), recall (-11.56%), and F1-Score (-11.70%). The Random Forest model with 500 n-estimators and 5 min-samples-split demonstrated a marginal improvement in accuracy (+0.06%) and significant gains in precision (+8.64%) and recall (+8.64%). Its F1-Score remained stable, further solidifying its position as the best-performing model for subfamily classification. Last but not least, the Decision Tree Classifier saw the biggest number of changes in its hyperparameter values, the criterion was changed to entropy, min-samples-leaf and min-samples-split also saw changes to their values of 5 and 10, respectively, and finally a max-depth of 20 was applies, with theses changes we saw an improvement in accuracy (+0.18%), precision (+9.61%), and recall (+9.60%). These results indicate that hyperparameter tuning enhanced the model's ability to capture patterns in the dataset.

Randomized Grid Search was then applied to try to improve the models' performance for the CIC-MalDroid-2020 dataset, the results are shown in Table VII.

TABLE VII
CLASSIFICATION RESULTS FOR THE CIC-MALDROID-2020 DATASET WITH MODEL'S HYPERPARAMETERS TUNNING

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.7866	0.7912	0.7340	0.7538
Random Forest	0.9340	0.9201	0.9232	0.9210
Decision Tree Classifier	0.8952	0.8717	0.8695	0.8706

When compared to the results without hyperparameter tunning we can observe that the Logistic Regression model once again barely changed any hyper parameters only choosing to increase the value of the regularization strength (C) to 235.69148616733443 and the number of max-iter to 1500, improving in terms of accuracy (+3.15%), precision (+3.10%), and F1-score (+0.79%) after hyperparameter tuning, obtaining a slight decline in recall (-2.11%), possibly due to the new hyperparameter settings emphasizing precision over recall. The Random Forest model performance remained largely unchanged after tuning, only changing its max-depth to 30, suggesting that the default hyper parameters were already well-suited for the dataset. For the Decision Tree Classifier, with changed to its criterion and max-depth, entropy and 20 respectively it managed to obtain a small improvement in accuracy (+0.26%), but declines in precision (-2.08%), recall

(-2.31%), and F1-score (-2.19%) were observed, suggesting that the tuning parameters might not have effectively captured the dataset's complexity.

V. Conclusion

This study highlights the effectiveness of supervised learning models, Logistic Regression, Random Forest, and Decision Tree, in malware classification across two distinct datasets: CIC-MalMem-2022 and CIC-MalDroid-2020. The results indicate the robustness of Random Forest and Decision Tree in both binary and multi-class classification, especially for handling complex relationships in data, while Logistic Regression served as a strong baseline for linear separability.

Key findings demonstrate near-perfect performance on the CIC-MalMem-2022 dataset for binary classification, validating its utility for memory-resident malware detection. However, the more challenging CIC-MalDroid-2020 dataset exposed limitations in model generalization, particularly for linear models like Logistic Regression, but highlighted the effectiveness of ensemble approaches such as Random Forest.

Compared to related studies, the models implemented in this work achieved competitive results. For example, previous research using the CIC-MalDroid-2020 dataset reported an accuracy of 97.46% with Random Forest combined with feature selection techniques [3]. In contrast, our study reached 93.40% accuracy using default feature sets, suggesting further improvements are possible.

Future work includes integrating deep learning models to capture more complex malware patterns, applying feature selection and dimensionality reduction to improve efficiency on high-dimensional data, and exploring real-time deployment scenarios.

References

[1] S. Mitra, S. A. Torri, and S. Mittal, "Survey of malware analysis through control flow graph using machine learning," 2023. [Online]. Available: https://arxiv.org/abs/2305.08993

[2] J.-F. Lalande, *Android Malware Analysis: From Technical Difficulties to Scientific Challenges: 11th International Conference, SecITC 2018, Bucharest, Romania, November 8–9, 2018, Revised Selected Papers*, 02 2019, pp. 17–21.

[3] D. Kshirsagar and P. Agrawal, "A study of feature selection methods for android malware detection," *Journal of Information and Optimization Sciences*, vol. 43, no. 8, pp. 2111–2120, 2022.

[4] R. Sihwail, K. Omar, and K. A. Zainol Ariffin, "An effective memory analysis for malware detection and classification," *Computers, Materials Continua*, vol. 67, pp. 2301–2320, 12 2020.

[5] K. M. Balasubramanian, S. V. Vasudevan, S. K. Thangavel, G. K. T, K. Srinivasan, A. Tibrewal, and S. Vajipayajula, "Obfuscated malware detection using machine learning models," in *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, 2023, pp. 1–8.

[6] T. Carrier, P. Victor, A. Tekeoglu, and A. Habibi Lashkari, "Detecting obfuscated malware using memory feature engineering," in *8th International Conference on Information Systems Security and Privacy*, 01 2022, pp. 177–188.

[7] H. Almajed, A. Alsaqer, and M. Frikha, "Imbalance datasets in malware detection: A review of current solutions and future directions," *International Journal of Advanced Computer Science and Applications*, vol. 16, no. 1, 2025. [Online]. Available: http://dx.doi.org/10.14569/IJACSA.2025.01601126

[8] S. Mahdavifar, A. F. Abdul kadir, R. Fatemi, D. Alhadidi, and A. Ghorbani, "Dynamic android malware category classification using semi-supervised deep learning," 08 2020, pp. 515–522.

[9] S. Mahdavifar, D. Alhadidi, and A. Ghorbani, "Effective and efficient hybrid android malware classification using pseudo-label stacked auto-encoder," *Journal of Network and Systems Management*, vol. 30, 01 2022.